CHAPTER 3

Hardware

Hardware

Inference engineering relies on accelerators: powerful hardware designed to load terabytes of data and perform trillions of operations per second.

The most common type of accelerator for inference is the GPU, and the market leader in GPUs for inference is NVIDIA. This book focuses on inference engineering for NVIDIA GPUs in the datacenter, but section 3.4 of this chapter covers other vendors of datacenter accelerators, and section 3.5 covers local inference.

Across vendors, there are three types of GPUs on the market:

- **Datacenter GPUs:** Racked servers with interconnected high-performance GPUs. Example: NVIDIA B200.
- **Workstation GPUs:** Individual desktop GPUs for professional workflows. Example: NVIDIA RTX Pro 6000.
- **Personal computing GPUs:** Individual desktop GPUs for everyday use. Example: NVIDIA GeForce RTX 5090.

Inference at scale uses datacenter GPUs mounted on racks: refrigerator-sized chassis with standardized power, networking, and cooling.

Datacenter GPUs like the NVIDIA B200 offer the highest individual performance, but more importantly, include high-bandwidth GPU-to-GPU interconnects, are installed in highly standardized configurations, and are available by the millions in datacenters worldwide.

I doubt that you have a B200 GPU running under your desk. If you do, send me a picture! Instead, inference on datacenter GPUs runs in one of three modes:

- **Cloud:** GPUs are rented in someone else's datacenter, usually hyperscalers like AWS and GCP or neoclouds like Coreweave and Nebius.
- **On-premise:** GPUs are purchased and installed in a datacenter that you control directly.
- **Air-gapped:** GPUs are installed on-premise and you need to physically access the GPUs to run inference.

Most inference engineers use cloud GPUs. Large enterprises and governments run on-premise and air-gapped deployments, but cloud-based GPUs offer the flexibility and access that fast-growing AI products need to scale.

Even with these constraints, navigating the hardware landscape is complex. From variations among cloud providers to NVIDIA's own naming conventions, there are many nuances in selecting the right accelerator.

3.1 GPU Architecture

GPUs are throughput machines. Where CPUs are great at complex sequential execution, GPUs are designed for simple, massively parallel workloads.

Specifically, GPUs are great at performing one uniform operation on thousands of independent pieces of data. Given that AI model inference is a series of vector and matrix multiplications, GPUs are a natural fit.

While the principle of highly parallel computation is simple, GPUs themselves are extraordinarily complex pieces of technology. Hardware engineering is a fascinating multidisciplinary field, from the physics of powering and cooling the chips to the impossibly tight tolerances involved in manufacturing each component.

Inference engineers work at a comfortable level of abstraction above the GPU hardware, but a strong mental model for what's going on inside the box is essential for building high-performance systems.

3.1.1 Compute

If you're familiar with CPUs, you've probably heard about cores, like an 8-core Intel i9 CPU in a high-end gaming computer.

In GPUs, cores have a different meaning. GPUs have Streaming Multiprocessors (SMs), while each SM contains multiple cores. There are three types of compute in GPUs:

- **CUDA Core:** Operates on individual numbers (scalars).

- **Tensor Core:** Operates on vectors and matrices.
- **Special Function Unit (SFU):** Accelerates certain mathematical operations like sin, cos, and log.

When measuring GPU compute for inference, measure in terms of Tensor Core compute. SFUs are essential for softmax, but Tensor Cores are responsible for Matrix Multiply and Accumulate (MMA) instructions, which are foundational to inference.

The "accumulate" step in MMA means adding the product of two matrices to a base matrix to produce the output, as shown in Figure 3.1.

Figure 3.1: Matrix Multiply and Accumulate (MMA) multiples matrix A by matrix B, adds matrix C, and stores the result as matrix D.

Unlike cores, the concept of a thread is similar between CPUs and GPUs. Where a CPU has dozens to hundreds of threads, GPUs have tens to hundreds of thousands of threads that can work concurrently, switch tasks in a single clock cycle, and execute simple instructions in parallel.

Compute is measured in FLOPS (floating point operations per second) and datacenter GPUs are capable of trillions or quadrillions of FLOPS (teraFLOPS and petaFLOPS, respectively). However, when you read a spec sheet, you'll see two measurements for Tensor Core compute:

- **Dense:** The raw floating-point operations per second if every element of the tensor is used.
- **Sparse:** In tensors with 2:4 structured sparsity, where 50 percent of the values are 0, Tensor Cores can skip multiplication by 0.

A GPU's FLOPS at a given precision with sparsity are often, but not always, double that of dense operations at the same precision. By default, inference is dense, so ensure that you're looking at FLOPS without sparsity.

FLOPS generally double with each halving of precision. A GPU capable of one petaFLOPS on 16-bit numbers will be able to do two petaFLOPS on 8-bit numbers. This is relevant for inference – be sure to compare FLOPS across GPUs at identical precisions.

Compute is the bottleneck for LLM prefill and for image and video generation. If you're selecting hardware with one of these use cases in mind, pick the accelerator with more FLOPS.

3.1.2 Memory and Caches

GPUs contain high-speed onboard memory called VRAM. Just like the "G" in GPU stands for graphics, the "V" in VRAM stands for video – a callback to these accelerators' original purpose.

Today, VRAM is added to GPUs in the form of HBM3, HBM3e, or HBM4, all various generations of high-bandwidth memory. GPUs have dozens or hundreds of gigabytes of VRAM.

There are two types of memory on any chip, CPU or GPU:

- **DRAM (Dynamic RAM):** General-purpose off-chip memory denominated in gigabytes.
- **SRAM (Static RAM):** Faster, more expensive, on-chip memory denominated in kilobytes or megabytes.

VRAM is a type of DRAM. GPUs also feature SRAM on-chip in the form of caches. GPUs have three levels of cache:

- **L0:** Instruction cache for a single Tensor Core.
- **L1:** Shared memory per Streaming Multiprocessor.
- **L2:** Global cache across Streaming Multiprocessors.

An H100 GPU has 256 KB of L1 cache per Streaming Multiprocessor and 50 MB total L2 cache on chip.

VRAM bandwidth measures the peak transfer rate between GPU cores and VRAM via the memory bus. In practice, this determines how quickly VRAM can supply data into the GPU's cache hierarchy.

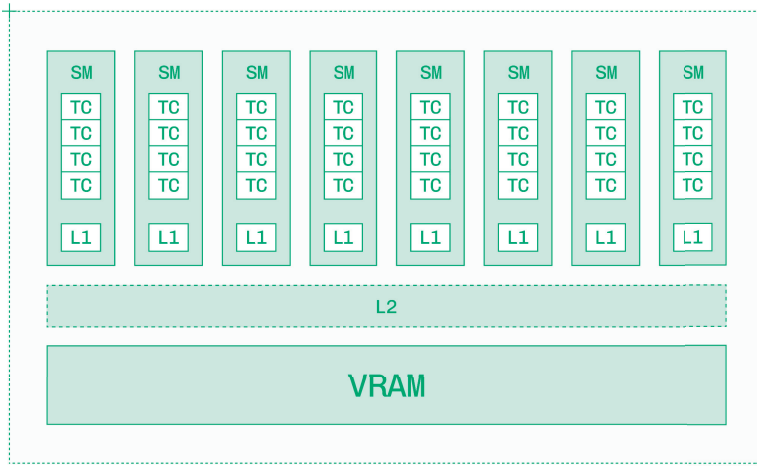

Figure 3.2: A GPU has multiple SMs, each of which has multiple Tensor Cores. L1 cache sits within SMs and L2 cache is shared.

The total amount of VRAM on a GPU limits the size of the model you can load onto it. The VRAM should hold the model weights, plus at least 50 percent headroom for KV cache (more for long context, high batch sizes, or video generation models).

If there isn't enough VRAM available for the weights, loading the model will fail with an OOM (out of memory) error. And if there isn't enough headroom, inference will be slow or crash with an OOM.

Memory bandwidth is the bottleneck for LLM decode at low to medium batch sizes. High-end GPUs have terabytes per second of memory bandwidth. If you are picking a GPU and want to generate more tokens per second, select the accelerator with a higher memory bandwidth, like the H200 instead of the H100.

3.2 GPU Architecture Generations

Hardware iteration cycles are slow. There are years of lead time between finalizing the architecture design and shipping GPUs.

Given the speed of the AI industry, this tapeout and testing process means that even next-generation GPUs were designed at a time when AI model

capabilities looked nothing like they do today. Designing a GPU architecture with staying power on the market requires foresight into how use cases will evolve over the expected lifetime of the GPU.

For years, training AI models was the primary use for GPUs. Now, with inference rising as the dominant use case, the latest architectures are introducing inference-focused features.

GPU names, like B200, have two parts:

- **Letter:** Signifies the architecture generation used in the chip.
- **Number:** Identifies individual models within the generation.

For example, the H100 directly replaces the previous generation A100, while the H200 is a larger GPU within the same generation (and is in turn supplanted by the B200).

The numbering appears somewhat arbitrary, with different sets of numbers used from generation to generation, but the general rule is that a bigger number means a larger, more powerful, more expensive GPU. On the other hand, the lettering for architecture is very meaningful.

Every one to two years, NVIDIA releases a new GPU architecture, which powers all of their products from the datacenter to personal computers. Each architecture generation introduces both improved base speeds on compute and memory and new features for more efficient inference.

Since 1998, NVIDIA has named their GPU architectures for prominent scientists.

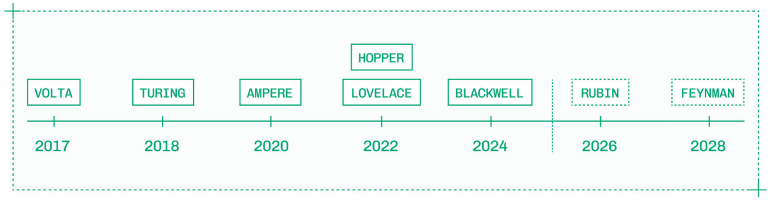

Figure 3.1: NVIDIA GPU architecture names from 2017 through all announced future architectures.

While GPU architectures go back decades, inference engineers generally work within the three to five most recent generations of GPUs. Even for cost-sensitive workloads, modern architectures' efficiency often makes them more cost-effective for large scale traffic, and of course newer architectures offer better performance.

You may still see Turing (T4) and Ampere (A10, A100) GPUs from time to time in low-traffic or legacy systems, but most deployments today use Lovelace (L4, L40), Hopper (H100, H200), or Blackwell (B200, B300) GPUs.

The Hopper and Blackwell architectures offer low-precision Tensor Cores, high-bandwidth memory, and inference-focused features, while Lovelace GPUs are used for low-cost inference on small models. The upcoming Rubin and Feynman architectures promise even greater performance when released in 2026 and 2028, respectively.

3.2.1 Hopper GPUs

GPU	FP8 compute (dense)	Memory	Bandwidth
H100	1,979 teraFLOPS	80 GB	3.35 TB/s
H200	1,979 teraFLOPS	141 GB	4.8 TB/s

The Hopper architecture, named for Rear Admiral Grace Hopper, was first released in March 2022 with the H100 GPU.

Hopper introduces support for FP8, a floating-point number format with 8 bits of precision. FP8 Tensor Cores are twice as fast as FP16 Tensor Cores, and moving FP8 values around takes half as much memory bandwidth. As section 5.1 discusses, this does not linearly translate to double the performance, but for workloads that can be run in FP8, it's a significant gain.

The Hopper architecture adds fourth-generation Tensor Cores within more and faster Streaming Multiprocessors than previous generations. Alongside dynamic programming instructions, thread block clusters, and distributed shared memory, Hopper GPUs give CUDA engineers more tools for writing high-performance kernels.

One such kernel is FlashAttention 3, which improves performance and memory efficiency for attention on Hopper GPUs. FlashAttention 3 takes advantage of the new asynchronous data transfer and execution features introduced with Hopper.

The H100 and H200 GPUs are among the most widely used inference accelerators, and for good reason. The Hopper architecture is new enough to be performant but established enough for industry-wide support and highly optimized kernels, and H100 and H200 GPUs are right-sized for common workloads across all modalities.

3.2.2 Ada Lovelace GPUs

GPU	FP8 compute (dense)	Memory	Bandwidth
L4	242 teraFLOPS	24 GB	300 GB/s
L40	362 teraFLOPS	48 GB	864 GB/s

The Ada Lovelace architecture, named for the first computer programmer, was first released just six months after Hopper.

The two architectures are similar, with Lovelace acting as more of a counterpart than a successor. Lovelace also supports FP8 inference.

Where Hopper is focused on AI applications, Lovelace GPUs are more graphics oriented. Lovelace GPUs do not support NVLink interconnect. This is a major limitation. Nodes with eight Hopper or Blackwell GPUs use these high-bandwidth interconnects for efficient parallelism. Lovelace GPUs must be used individually or via inefficient parallelism methods like Pipeline Parallelism.

L4 GPUs can be a cheap and convenient way to run small models for modalities like text embeddings and computer vision.

But L40 GPUs generally aren't a great choice for inference. For the same memory footprint, multi-instance GPUs (section 3.3.2) offer much higher compute and memory bandwidth on fractional H100s.

3.2.3 Blackwell GPUs

GPU	FP8 compute (dense)	Memory	Bandwidth
B200	~5 petaFLOPS	192 GB	Up to 8 TB/s
B300	~5 petaFLOPS	288 GB	Up to 8 TB/s

The Blackwell architecture, named for mathematician David Blackwell, was first released in November 2024 with the B200 GPU, followed by the B300. While the B100 does exist, it's not common for inference.

Where Hopper introduced FP8, Blackwell goes further in low-precision computing with FP4, a 4-bit floating point format, plus a set of microscaling formats (MXFP8, MXFP4, NVFP4) for better quality retention during inference. Section 5.1 explains these formats in detail.

Blackwell builds on Hopper's asynchronous programming paradigm with more features for loading and storing between tensor and global memory. The updated FlashAttention 4 kernel relies heavily on tiling loads, computations, and writes in asynchronous pipelines.

The B200 and B300 are the new gold standard for inference, offering the highest performance for large language models and demanding workloads like video generation. Software support, optimized kernels, and general availability for Blackwell GPUs have all come online in recent months, marking an important transition in the inference industry.

3.2.4 Rubin GPUs

The Rubin architecture, named for astronomer Vera Rubin, will launch in 2026 as NVIDIA's next-generation GPU architecture.

When evaluating new architectures, it's important to reserve judgement until you can run real-world performance benchmarks. New architecture rollouts, followed by industry-wide software support, take a year or so to fully saturate.

At the time of publication, there are some concrete details about Rubin. The Rubin architecture uses HBM4, an upgrade from the HBM3 and HBM3e that has powered the last few generations of GPUs. Inference

tasks like LLM decode that are bound on memory bandwidth will benefit from this higher-throughput VRAM.

Rubin also introduces the new CPX, a separate chip that's built for compute-bound tasks like LLM prefill. The CPX will be part of NVIDIA's rack-scale systems for high-volume inference.

After Rubin, NVIDIA will release Feynman in 2028. Few details are known about Feynman, but it is likely to support larger and more powerful chips with a faster memory architecture.

3.2.5 Grace and Vera CPUs

NVIDIA also offers its own ARM-based CPUs, which are integrated with their GPUs on superchips like the GH200 and GB200. The "G" stands for "Grace," as in Grace Hopper (to match Hopper GPUs).

NVIDIA Grace GPUs have overall strong compute performance, but what matters for inference is that they have a much higher-bandwidth connection between the CPU and GPU. Grace CPUs use NVIDIA NVLink Chip to Chip for up to 900 GB/s bi-directional bandwidth between the CPU and GPU memory.

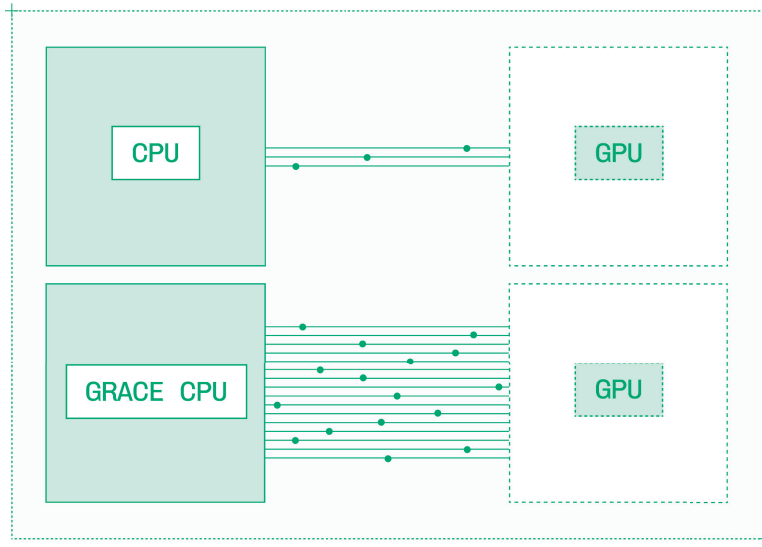

Figure 3.3: Grace CPUs have a higher-bandwidth interconnect to the GPU than standard CPUs do.

This CPU-to-GPU interconnect is several times faster than PCIe or other standard connections between CPUs and GPUs.

Some inference setups call for offloading important information like LoRA fine-tune weights and KV caches from previous inference calls to CPU memory, which is far larger than GPU memory. With Grace CPUs, this information can be retrieved much faster.

For the Rubin architecture, the Vera CPU (named for Vera Rubin) replaces the Grace CPU that was used on Hopper and Blackwell systems.

3.3 Instances

The atomic unit of GPU allocation on the cloud is an instance. An instance is a virtual machine that includes:

- **GPUs (device):** One or more GPUs for inference tasks.
- **CPUs (host):** General-purpose compute for any tasks that don't run on the GPU.
- **Memory (host memory):** General-purpose memory for CPU operations (traditional RAM).
- **Storage:** Disk memory for loading and storing large files.
- **Networking:** Physical network connections to the datacenter and eventually the public internet.
- **Interconnect:** Physical GPU-to-GPU and node-to-node connections for running on multiple GPUs at once.

Not all GPUs, and not all instances, are created equal. Depending on your cloud provider, instances vary in compute, memory, storage, and interconnect. While NVIDIA offers its own reference architectures, each cloud provider ultimately builds out systems according to their own preferences.

Even the GPU itself can differ from instance to instance. For example, NVIDIA A100 GPUs have two form factors: PCIe and SXM. But most inference on A100 runs on SXM GPUs as they have five percent higher memory bandwidth than the PCIe variant.

When provisioning instances, it's essential to understand exactly what you're getting. Any component of the instance, not just the GPU, could present a bottleneck or failure point during inference.

3.3.1 Multi-GPU Instances

Often, a model is too big to run on a single GPU, or inference engineers want to use multiple GPUs together to improve performance. It's common to need two, four, eight, or even more GPUs to run large models like DeepSeek or demanding modalities like video generation.

The standard unit for GPUs is a node, which contains eight individual GPUs. For example, a B200 node contains eight B200 GPUs.

These GPUs are connected together via two systems:

- **NVLink:** A one-to-one communication layer between GPUs, up to 1800 GB/s on Blackwell and 900 GB/s on Hopper.
- **NVSwitch:** An all-to-all communication layer on top of NVLink for coordination among all GPUs in a node.

These high-bandwidth interconnects make it possible to spread inference on a single model across multiple GPUs, up to a full 8-GPU node.

But sometimes one node isn't enough. Running extremely demanding inference workloads on more than eight GPUs requires a high-bandwidth interconnect between nodes.

The standard in node-to-node interconnect for NVIDIA GPUs is InfiniBand. InfiniBand competes with networking technologies like Ethernet, and in 2019 NVIDIA bought Mellanox, which manufactures InfiniBand.

InfiniBand is much slower than NVLink, with specs up to 400 Gb/s per Network Interface Controller (NIC). But it's the fastest node-to-node interconnect on the market – Ethernet maxes out at 100 Gb/s per NIC.

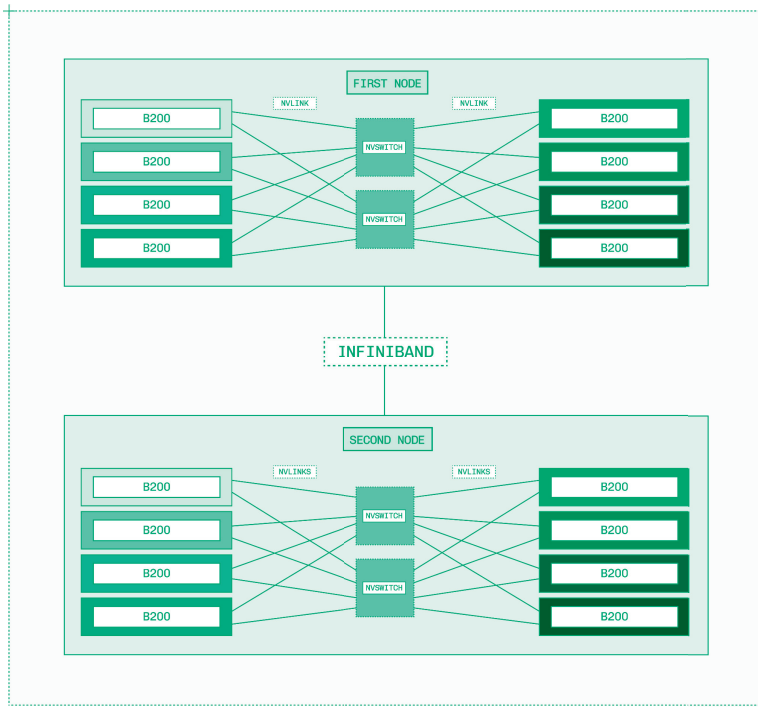

Figure 3.4: NVLink, NVSwitch, and Infiniband work together to enable GPU-to-GPU communication.

Not every cloud provider uses InfiniBand. Some offer their own interconnects, while others have InfiniBand on some GPUs but not all. When provisioning GPUs, double-check what interconnect is provided and what bandwidth that interconnect delivers.

In addition to InfiniBand, NVIDIA offers a high-end solution for NVLink connections among more than eight GPUs. Their NVL72 GB200 system combines 72 Blackwell GPUs and 36 Grace CPUs on a full-rack system. These systems provide massive throughput for serving the world's largest models with intense traffic.

The NVIDIA Vera Rubin NVL 144 CPX is the next generation of this massive system, with updated Vera CPUs and Rubin GPUs alongside the new Rubin CPX.

When working with multi-GPU and multi-node systems, keep relative bandwidths in mind. When an interconnect like NVLink is an order of magnitude faster than InfiniBand, it can handle far more data before becoming a bottleneck. Parallelism and disaggregation techniques discussed in sections 5.4 and 5.5 navigate this topology to deliver performant inference across multiple GPUs.

3.3.2 Multi-Instance GPUs

Sometimes, inference engineers run into the opposite problem: the GPU is too big for the model.

Using newer high-performance architectures like Hopper and Blackwell requires GPUs like the H100 or B200, with relatively large compute and memory allocations. For models with a couple of billion parameters or fewer, it's hard to utilize these GPUs effectively. Even with large batch sizes, valuable GPU resources are wasted.

Rather than running small models on older, lower-performance GPUs, there's a way to run these lightweight workloads on fractions of newer, high-performance GPUs.

Multi-instance GPU (MIG) is a hardware-level capability in larger GPUs including the A100, H100, H200, and B200. These GPUs can be split into as many as seven pieces. These fractional GPUs also receive a slice of CPU, RAM, storage, and other resources required to form an instance.

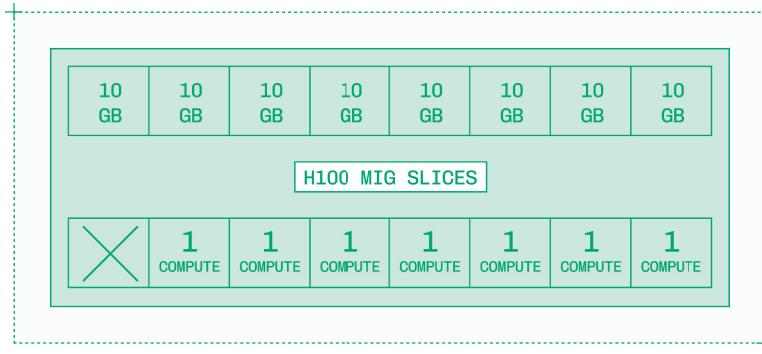

Figure 3.5: An H100 has eight memory slices and seven compute slices for assembling multi-instance GPU instances.

For example, an H100 MIG with three slices has about 3/7 of the available compute and can access as much as half of the total VRAM, or 40 GB. It also includes about half of the CPU cores, CPU memory, storage, and network bandwidth allocated to the underlying H100 to form an instance.

Software engineering generally works in multiples of two, so seeing seven compute slices may appear strange. Compute slices are made up of Streaming Multiprocessors in a GPU. GPUs generally do not have a clean multiple-of-two SM count, for example, an SXM H100 GPU has 132 SMs. Accordingly, seven evenly-sized compute slices are created, and the leftover SMs are left idle.

For small models like Orpheus TTS, a 3B parameter model, using two MIG instances is sometimes a more efficient use of resources than allocating the full GPU to a single instance.

3.4 Other Datacenter Accelerator Options

NVIDIA's leadership in the AI hardware market has made it the world's most valuable company. But it is far from the only company to make hardware that can run AI inference.

From fellow industry giants like Amazon and Google to a massive crop of startups, competitors are pouring billions of dollars into developing and manufacturing alternatives to NVIDIA GPUs.

While this book focuses on optimizing inference on NVIDIA GPUs, here's a short list of the other notable hardware options for running model inference.

Company	Stage	Flagship Product
AMD	Public	**MI350 GPU:** A datacenter GPU with competitive specs on AMD's own software stack.
AWS	Public	**Inferentia** and **Trainium:** A pair of chips purpose-built for inference and training, respectively, on AWS.
Cerebras	Startup	**WSE-3:** A wafer-scale chip with extremely high memory bandwidth to remove decode bottlenecks.

Company	Stage	Flagship Product
Etched	Startup	**Sohu:** An Application-Specific Integrated Circuit (ASIC) for the transformer architecture.
Furiosa	Startup	**RNGD:** A power-efficient accelerator designed for tensor contraction operations.
Google	Public	**TPU:** A Tensor Processing Unit is an AI-specific ASIC built for inference and training.
Groq	Startup	**LPU:** A composable language processing unit relies on SRAM for high memory bandwidth
Qualcomm	Public	**Cloud AI 100 Ultra:** A full-sized GPU composed of multiple power-efficient mobile GPUs.
Sambanova	Startup	**RDU:** A Reconfigurable Dataflow Unit with large memory allocation for trillion-parameter models.

GPUs are fairly general-purpose accelerators. Every hardware company competing to win inference workloads from NVIDIA is betting on a specific edge where their product can win:

- **Memory bandwidth:** Startups like Cerebras and Groq achieve high token per second scores for LLMs by accelerating decode on ultra-high-bandwidth memory.
- **Power efficiency:** Companies like Furiosa and Qualcomm design chips for lower power consumption, which leads to cheaper operating costs.
- **Platform integration:** Enterprises like Amazon and Google build deep integrations with their cloud service platforms and proprietary closed models.

While each of these accelerator options have their winning use cases, they share common challenges:

- **Software:** Without CUDA, hardware providers have to rebuild the entire inference stack for their accelerators.
- **Manufacturing:** Companies need to assemble the most complex object humankind has ever created.
- **Distribution:** After manufacturing a chip, providers need it installed and brought online to put capacity on the market.

Competition accelerates innovation. A market with more hardware options in the datacenter is good for every inference engineer. Competition in this space will only grow more robust as inference workloads become more valuable, as will exploration of options outside the datacenter such as local inference.

3.5 Local Inference

Local inference, also called edge inference, client-side inference, or on-device inference, means running AI model inference directly on the end user's device rather than on a centralized server.

Client-side inference has four massive advantages over server-side inference:

- **Zero network latency:** There's no communication overhead, saving tens or even hundreds of milliseconds.
- **Independence:** There's no dependency on internet connection or impact from high server traffic or downtime.
- **Improved privacy:** The end user's data never leaves their device.
- **Cost:** Datacenter GPUs are expensive, while edge inference is free for the developer, unlocking new business models.

Local inference sounds perfect in theory. In practice, there's a reason most inference happens in the cloud. There are four weaknesses to local inference that limit its applications:

- **Hardware capabilities:** Even high-end prosumer desktops offer a fraction of the speed and power of datacenter GPUs.
- **Thermal constraints:** Local devices have worse cooling than datacenters, further limiting their speed and power.
- **Fragmented support matrix:** Endless combinations of hardware and software make standardization challenging.
- **Battery life:** Inference is a demanding workload that quickly drains the batteries of laptops and smartphones.

When building with local inference, it's important to keep your audience in mind. An AI enthusiast may have the latest and greatest phone and a

powerful computer, but a median user is more likely to have older devices with less powerful components.

Local inference is turning the corner from experimentation to production, with a strong ecosystem across hardware and software and a vibrant community closely affiliated with the world of open models.

3.5.1 Desktop Inference

The classic local device is a workstation or gaming PC equipped with one or two high-end consumer GPUs from NVIDIA or AMD. While researchers and enthusiasts do use these setups, they're a small portion of the desktop inference market.

Increasingly, Apple is the leader in the desktop inference market. Apple's custom M-series CPUs and GPUs draw from a single unified memory, giving inference on GPUs access to far more memory, albeit at slower speeds.

The highest-end options from Apple and NVIDIA currently available on the market illustrate the tradeoff between memory capacity and speed.

Hardware	**NVIDIA RTX 5090**	**Apple M3 Ultra**
Memory	32 GB	512 GB
Bandwidth	1,792 GB/s	819 GB/s
Cost (full computer)	$5,000	$10,000

This trend continues through midrange hardware at more reasonable price points. Low-end computers like Chromebooks are not equipped to run any meaningful local inference.

The enthusiast-led open-source ecosystem focuses on running frontier open models on desktops and laptops. Today, running aggressively quantized 100B+ parameter models on high-end personal hardware is possible using tools like Ollama and llama.cpp.

The increased popularity of Mixture of Experts is also a tailwind for desktop inference. These models have fewer active parameters, meaning that

an individual user's single request only touches a fraction of the model's total weights.

Image generation is also quite popular on personal computers, especially via ComfyUI, a tool for assembling multiple image model components together into a single pipeline.

Smaller language models and other modalities like speech are also feasible to run on midrange computers, and the industry is rapidly developing browser inference libraries like WebLLM and other cross-platform standards to bring these capabilities from early adopters to the mainstream.

3.5.2 Mobile Inference

Local inference on mobile devices represents the majority of on-device workloads today. Both major operating systems offer tooling for developers to add edge inference to applications:

- **Android:** Google's AI Edge SDK and ML Kit GenAI APIs interface with Gemini Nano and OSS Gemma models.
- **iOS:** Apple's Foundation Models and Core ML frameworks provide APIs for models across modalities.

Mobile devices have extremely limited hardware capabilities and battery capacities, making inference even more challenging. Even high-end phones struggle to run models with more than one or two billion parameters.

Still, some modalities are well-suited for edge inference on phones. For example, transcription and speech synthesis models are latency-sensitive, and some models are small enough to run in real time on modern phones. Other discrete tasks, like translation, can be handled on edge devices by small fine-tuned models.

Like any other software, the future of inference isn't local or cloud, it's both working together to power fast and seamless user experiences. Small models and quick queries will run on end-user devices, while more demanding workloads will remain on datacenter GPUs in the cloud.

CHAPTER 4

Software

Software

NVIDIA's market dominance in the inference space is in no small part due to the robust and mature software ecosystem around its hardware.

Hardware iteration cycles are slow. Best-in-class hardware companies like Apple and NVIDIA release new architectures and generations at most yearly, with two-year release cycles being more common.

But software iteration is fast. Often, to run a newly released open model on day zero, you need to install a nightly build or other prerelease version of each of your software dependencies just to get support for the new model.

Software's fast iteration cycle and lower barrier to entry dramatically expands the landscape of inference engineering. While hardware centers on NVIDIA and a few competitors, there are countless companies building software at various levels of the inference stack.

For inference engineers, these are some key players:

- **NVIDIA:** Invests heavily in its own sometimes-proprietary software ecosystem, from CUDA up to Dynamo.
- **Hugging Face:** Maintains a model registry for all open models plus `transformers` and `diffusers`.
- **The Linux Foundation:** Maintains hardware-agnostic projects like PyTorch and vLLM.
- **LMSYS Org:** Develops essential tools for inference and evaluation, most notably SGLang.

There are thousands more companies, universities, and research institutions making essential open-source contributions to inference.

The software space is too big and too fast to exhaustively document in any book, much less in a single chapter. Instead, this chapter presents foundational technologies with long-term relevance.

Throughout the chapter, technologies are presented with increasing levels of abstraction:

- **CUDA:** Direct communication to the GPU for explicit control over computations and memory (section 4.1).
- **Deep learning frameworks:** Abstractions over CUDA for training, exporting, and running neural networks in Python (section 4.2).
- **Inference engines:** Highly configurable PyTorch-backed inference for common architectures (section 4.3).
- **NVIDIA Dynamo:** Sits on top of inference engines to power large-scale deployments (section 4.4).

Most inference engineering today happens at the higher levels of abstraction, configuring and deploying inference engines and orchestrating inference across multiple GPUs. No matter what level of the stack you work at, it's essential to have a strong mental model for the adjacent levels of abstraction to guide your work.

4.1 CUDA

CUDA is how you write code to run on NVIDIA GPUs.

More formally, CUDA is NVIDIA's proprietary computing platform and programming model for executing parallel tasks on GPUs. Both "platform" and "programming model" are broad definitions; it's easier to look at CUDA in its component parts:

- **CUDA kernel:** A user-defined function that executes parallelized code on the GPU.
- **CUDA graph:** A directed acyclic graph (DAG) of kernels and other GPU operations for optimizing repeated workflows.
- **CUDA driver:** A low-level interface between the application and the GPU hardware to manage memory and execution.
- **CUDA runtime:** A developer-facing API for launching kernels and managing memory.

CUDA – which stands for Compute Unified Device Architecture, though the acronym is rarely expanded today – is the foundation for the entire generative AI ecosystem on NVIDIA GPUs.

CUDA is not a programming language. Instead, CUDA programs are specified in a programming language, most often C++, then compiled into separate CPU and GPU code by a compiler like `nvcc`.

A kernel is just a function that does some parallel computation. Whenever you see the phrase "CUDA kernel" you can replace it with "a piece of code written for NVIDIA GPUs."

For a "Hello, World!" kernel, consider these six lines of C++ code which take in an array of length `n` and double each element in the array.

```
__global__ void doubleArray(float *arr, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        arr[i] *= 2.0f;
    }
}
```

Figure 4.1: Example CUDA kernel doubles each value in an array.

Ordinarily, on a CPU, this function would run in linear time, with each element in the array being doubled sequentially. But on a GPU, thousands of elements can be processed simultaneously, making this function much more efficient.

Writing CUDA kernels shifts inference engineering from thinking about algorithms to thinking about implementations. For example, the traditional attention algorithm that is central to generative AI can be expressed in a few dozen lines of code. However, FlashAttention, which is the same mathematical operation, takes tens of thousands of lines of code to implement the algorithm in a more memory-efficient manner for a specific GPU.

4.1.1 CUDA Kernels for Inference

Writing CUDA kernels does not mean building from scratch.

The prior art to build upon predates CUDA by decades. BLAS (Basic Linear Algebra Subprograms), first implemented for Fortran in the 1970s, is a specification for common linear algebra operations from dot products to matrix multiplication.

cuBLAS, the CUDA implementation for BLAS, brings this specification to CUDA in the form of pre-built kernels for essential linear algebra operations. Similarly, cuDNN (CUDA Deep Neural Network) provides primitives for neural networks.

Within BLAS, the most frequently used operation in inference is GEMM (General Matrix-Matrix Multiplication). Every linear layer in a model uses matrix multiplication, and cuBLAS provides a strong starting point.

But you aren't limited to the cuBLAS implementation. As operations like GEMM are so essential for inference, you may need more fine-grained control. You might write different GEMM kernels for matrices of different shapes or to better run on specific GPU architectures.

CUTLASS is a template library that provides building blocks for writing high-performance kernels. For example, FlashAttention 3 uses CUTLASS. CuTe, another template library, introduces abstractions for tiled tensor operations for recent architectures. With tools like CUTLASS and CuTe, kernels can be written at a higher level of abstraction while retaining strong performance.

Another source for kernels is FlashInfer, a library with high-performance implementations of kernels for LLM inference, including many optimized attention kernels and fused sampling functions.

4.1.2 CUDA Kernel Selection

Most inference engineers will never need to write their own kernels. However, kernel selection – choosing the best kernel from a range of options – is an important part of inference optimization.

Kernel implementations are highly specialized. CUDA exposes low-level APIs for memory management and parallel computing, and kernel engineers make implementation decisions based on the exact specifications of the hardware they're writing for.

It's important to understand how closely tied kernel implementations are to specific hardware details. Kernels often have hard-coded values based on the memory bandwidth or the number or layout of Tensor Cores on a given GPU.

A kernel written for an H100 will likely fail to take advantage of the architecture and extra memory of a B200, while a kernel written for that B200 could be backwards incompatible with the previous-generation Hopper architecture. With each generation of GPUs, porting handwritten kernels to run optimally on the new architecture takes substantial engineering work.

Most kernel selection is automatic. Deep learning frameworks and inference engines have pre-configured kernels for various architectures, while PyTorch and TensorRT-LLM include automatic kernel selection in their compilation steps.

However, you might manually choose a few kernels for essential algorithms to speed up inference.

For example, most production-ready GEMM kernels come from cuBLAS. But when the DeepSeek lab released their updated version of DeepSeek-V3, they also released DeepGEMM, which provides more efficient GEMM kernels for running matrix multiplication in FP8 on the Hopper GPU architecture.

Manual kernel selection lets you insert a DeepGEMM kernel as a plugin to speed up a specific step in inference, like multiplying two matrices of precise dimensions. Just be careful to ensure compatibility. For example, if you were to upgrade to a B200 GPU, you'd have to either swap the kernel back, wait for Blackwell support in DeepGEMM (now supported as of the time of publication), or port the kernel yourself.

4.1.3 Reducing Memory Accesses with Kernel Fusion

Running two different kernels back-to-back on the same data results in wasted reads from and writes to memory.

As a simple example, imagine two kernels: `multiply_by_2` and `multiply_by_3`. If these kernels were run back-to-back, the sequence would be:

1. Read the input vector `[1, 2, 3]` from memory.
2. Run `multiply_by_2` on the vector.
3. Save the output vector `[2, 4, 6]` to memory.
4. Read the new input vector `[2, 4, 6]` from memory.
5. Run `multiply_by_3` on the vector.
6. Save the final output vector `[6, 12, 18]` to memory.

The inefficiency is clear: steps three and four comprise an unnecessary round trip to memory. During decode, the bandwidth-bound phase of LLM inference, an inference engine can't afford unnecessary reads from or writes to memory.

Kernel fusion is the process of taking two or more kernels and re-implementing them into a single kernel that handles both operations. In this example, the fused kernel would be `multiply_by_6` and the new order of operations would be:

1. Read the input vector `[1, 2, 3]` from memory.
2. Run `multiply_by_6` on the vector.
3. Save the final output vector `[6, 12, 18]` to memory.

In practice, kernel fusion is far more complicated – functions are more complex, and data overlap isn't as clean. But there are common patterns in kernel fusion for inference, like combining matrix multiplication, bias adding, and activation.

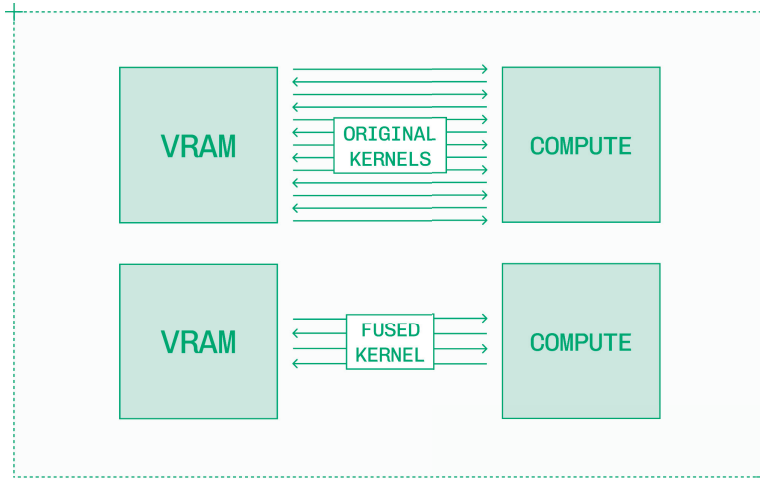

Figure 4.2: Kernel fusion reduces reads and writes between memory and compute within a GPU.

Kernel fusion can be an automatic or a manual process. Compilers can identify straightforward fusion opportunities and automatically create fused kernels. But more sophisticated algorithms, like FlashAttention, require handwritten fused kernels, which are used via plugins during inference.

4.2 Deep Learning Frameworks and Libraries

Deep learning frameworks and libraries are the bridge between working directly in CUDA and using off-the-shelf inference engines like vLLM. These libraries are used in both training and inference.

Over the past few years, PyTorch has emerged as the clear leader at this level of the stack. There are two other frameworks that, for concision, I will only mention briefly:

- **TensorFlow:** An end-to-end machine learning platform officially supported by Google, TensorFlow was prominent in the 2010s ML era but has fallen out of favor today.
- **JAX:** A research project unofficially associated with Google, JAX presents a simpler interface without as many legacy features and operations. However, as the documentation warns, expect sharp edges.